MODULE 30 · EXAM STRATEGY - ALL DOMAINS

Confusion pairs

Sixty-six look-alike pairs, each with the tiebreaker that separates it from its neighbour.

SECTION Exam skills COVERS All domains MODULE 30 of 35

admin@sthithapragna.com

Most wrong answers are the right answer's neighbour. Learn the one fact that tells them apart.

admin@sthithapragna.com

How to use this module

- **Each row is a pair** and its tiebreaker, the fact that decides between them.
- **Questions plant a detail** that points to one side: 'must not restart', 'every node', 'one namespace'.
- **Find the planted detail first,** then apply the tiebreaker.
- **Replay this module** the day before the exam. It is the course in about twelve minutes.

> ON THE EXAM
>
> The planted detail plus the tiebreaker answers most pair questions in seconds.

Fundamentals (1): containers and runtimes

PAIR	TIEBREAKER
Container / virtual machine	Container shares the host kernel; a VM brings its own
Namespaces / cgroups	Namespaces isolate what it sees; cgroups limit what it uses
CPU limit / memory limit	Over CPU: throttled. Over memory: OOMKilled
CRI / OCI	CRI: kubelet to runtime. OCI: image and runtime formats
containerd / runc	containerd pulls and manages; runc just starts the process

ON THE EXAM

Modules 1 and 2. The shared kernel explains most container questions.

Fundamentals (2): objects and workloads

PAIR	TIEBREAKER
spec / status	You write spec; the system writes status
ReplicaSet / Deployment	ReplicaSet keeps a count; Deployment manages versions
Deployment / StatefulSet	Identical copies; or named individuals with own storage
DaemonSet / Deployment	One per node, following nodes; or a count anywhere
Job / CronJob	Runs once to completion; or on a schedule
create / apply	create fails if it exists; apply is idempotent

ON THE EXAM

Identical, individual, everywhere, finite: the workload shortcut from module 8.

Fundamentals (3): admin and scheduling

PAIR	TIEBREAKER
cordon / drain	Close to new bookings; or also move the diners out
requests / limits	Requests decide placement; limits decide punishment
Taint / node affinity	Taint keeps others out; affinity pulls yours in
Required / preferred affinity	Required filters (Pending if unmet); preferred scores
Labels / annotations	Labels are selected on; annotations only describe
ResourceQuota / LimitRange	Quota caps the namespace; LimitRange shapes each container

ON THE EXAM

A toleration permits a tainted node; it never attracts to it.

Orchestration (1): Services and traffic

PAIR	TIEBREAKER
ClusterIP / NodePort	Inside the cluster only; or a port on every node
NodePort / LoadBalancer	You bring the front door; or the cloud builds one
Headless / ClusterIP	DNS returns Pod IPs; or one balanced virtual IP
Ingress / Gateway API	Frozen, one object; or role-split and expressive

ON THE EXAM

'Every Pod individually' is headless; 'one entrance for many Services' is Ingress or Gateway.

Orchestration (2): security and failures

PAIR	TIEBREAKER
Role / ClusterRole	One namespace; or cluster-wide and reusable
RoleBinding / ClusterRoleBinding	The binding decides the scope
NetworkPolicy / mTLS	Restricts who connects; or encrypts and authenticates
OOMKilled / Evicted	One container over its limit; or whole Pod, node pressure

ON THE EXAM

A ClusterRole bound by a RoleBinding only applies in that namespace.

Delivery (1): pipelines and releases

PAIR	TIEBREAKER
CI / CD	Build and test an artefact; or move it into environments
Continuous delivery / deployment	A human gate before production; or none
Blue-green / canary	Everyone at once, instant rollback; or a few users first
Rolling update / Recreate	Gradual, no downtime; or all out, then all in
Build once / rebuild per environment	Promote the same image; rebuilds can differ

ON THE EXAM

'Must never overlap' is Recreate; 'a few real users first' is canary.

Delivery (2): Helm and Kustomize

PAIR	TIEBREAKER
Helm / Kustomize	Templates and packages; or patches on plain YAML
Chart / release	The package; or one installed copy of it
Release / revision	One installed copy; or a numbered step in its history
helm template / helm install	Renders locally; or installs into the cluster
-f values file / --set	Files, in order; --set wins over both
Flux Kustomization / kustomization.yaml	A Flux custom resource; or Kustomize's own file

ON THE EXAM

Someone else's software: Helm. Your own app, plain YAML: Kustomize.

Delivery (3): GitOps

PAIR	TIEBREAKER
Push / pull deployment	Pipeline holds cluster credentials; or an agent pulls from Git
Argo CD / Flux	Application objects and a UI; or a toolkit of controllers
prune / selfHeal	Deletes what Git removed; or reverts manual changes
Synced / Healthy	Cluster matches Git; or the app actually works
git revert / kubectl rollout undo	Sticks under GitOps; or is reverted as drift
Application / ApplicationSet	One app; or a template generating many

ON THE EXAM

Synced but Degraded: Git was applied faithfully, and Git is wrong.

Delivery (4): probes and debugging

PAIR	TIEBREAKER
Liveness / readiness	Restart me; or do not send me traffic
Startup / liveness	Hold checks back while starting; or check once running
kubectl exec / kubectl debug	Use tools in the image; or bring your own
logs -f / logs --previous	Follow the live container; or read the one that crashed
get / describe	What state; or why, with events

ON THE EXAM

If a restart would not fix it, it belongs in readiness, not liveness.

Architecture (1): observability

PAIR	TIEBREAKER
Metrics / logs / traces	Something is wrong; what exactly; which service
Prometheus / Grafana	Collects and stores; or draws dashboards
metrics-server / Prometheus	Current CPU and memory for top and HPA; or history
Counter / gauge	Only goes up; or goes up and down
Histogram / summary	Buckets, percentiles at query time; or in the client
Scrape / Pushgateway	Prometheus pulls; short jobs push instead

ON THE EXAM

Never graph a raw counter. rate() turns it into per second.

Architecture (2): standards, SLOs and scaling

PAIR	TIEBREAKER
OpenTelemetry / OpenTracing	Current and graduated; or archived
SLI / SLO / SLA	A measurement; a target; a contract
HPA / VPA	More Pods; or bigger Pods
HPA / Cluster Autoscaler	Pods overloaded; or Pods Pending for lack of room
HPA / KEDA	CPU or memory; or events, down to zero
Monolith / microservices	One deployable; or many, independently released

ON THE EXAM

Pending with Insufficient cpu is a node problem, not a Pod-count problem.

Architecture (3): community

PAIR	TIEBREAKER
TOC / Governing Board	Technical decisions and maturity; or budget and business
Sandbox / incubating	Early and experimental; or growing production use
Incubating / graduated	Growing; or broad adoption plus a security audit
SIG / working group	Owns an area long-term; or a temporary goal
KEP / pull request	Designs a significant change; or implements code
Platform engineer / SRE	Builds the paved road; or keeps it reliable
CNCF / OCI	Both Linux Foundation; the OCI is not a CNCF project

ON THE EXAM

Project maturity is always decided by the CNCF's TOC.

Eight pair questions, six from Domains 3 and 4. In each, **find the planted detail** first.

> During a security investigation, node-6 must accept no new Pods for the next hour.
>
> The Pods already on it must keep serving customers throughout.

Which command fits?

A kubectl drain node-6

B kubectl cordon node-6

C A NoExecute taint on node-6

D kubectl delete node node-6

◆ PAUSE AND DECIDE

Which command fits?

A kubectl drain node-6

Also evicts the Pods that must keep serving.

B kubectl cordon node-6

No new Pods; existing ones stay.

C A NoExecute taint on node-6

Evicts running Pods without tolerations.

D kubectl delete node node-6

Removes the node from the cluster.

THE SIGNAL

No new Pods, existing ones stay: cordon.

> The security team requires traffic between the payment worker and the booking API to be encrypted in transit.
>
> Only those two services talk to each other already.

What meets the requirement?

A. A NetworkPolicy allowing only that pair

B. A ClusterIP Service between them

C. A headless Service for the booking API

D. Mutual TLS, typically through a mesh

◆ PAUSE AND DECIDE

What meets the requirement?

A A NetworkPolicy allowing only that pair
Restricts who connects; encrypts nothing.

B A ClusterIP Service between them
Addresses traffic; does not encrypt it.

C A headless Service for the booking API
Changes DNS answers; no encryption.

D Mutual TLS, typically through a mesh
Encrypts and authenticates each call.

THE SIGNAL

Encrypted in transit: mTLS. NetworkPolicy never encrypts.

> The fare engine container crashed and has just been restarted. kubectl logs shows only: Starting up...
>
> The engineer needs the error the container printed just before it crashed.

Which flag gives that?

A --previous

B -f

C --tail=100

D --since=1h

◆ PAUSE AND DECIDE

Which flag gives that?

A `--previous`

The last instance: the one that crashed.

B `-f`

Follows the new container, which has not failed.

C `--tail=100`

Recent lines of the current container only.

D `--since=1h`

A time filter on the current container only.

THE SIGNAL

Output from before a crash: `--previous`.

> Quayside's new search feature must be tried on real traffic from 2% of users for a week.
>
> Only if error rates stay low should everyone get it.

Which release strategy fits?

A Blue-green, switching all users at once

B Recreate, stopping the old version first

C A canary, starting at 2% of users

D A rolling update with default settings

◆ PAUSE AND DECIDE

Which release strategy fits?

A Blue-green, switching all users at once
Moves everyone together; no 2% phase.

B Recreate, stopping the old version first
Everyone at once, with downtime.

C A canary, starting at 2% of users
A small share first, expanded only if healthy.

D A rolling update with default settings
Replaces every Pod within minutes.

THE SIGNAL

'A few real users first, then everyone': canary.

> Quayside wants a metric for the number of bookings currently waiting in payment processing.
>
> The number rises when orders arrive and falls as they complete.

Which metric type fits?

A Counter

B Histogram

C Gauge

D Summary

◆ PAUSE AND DECIDE

Which metric type fits?

A **Counter**
Only goes up; cannot fall as bookings complete.

B **Histogram**
Distributions of observations, such as latency.

C **Gauge**
Goes up and down: a current level.

D **Summary**
Client-side percentiles, not a current level.

THE SIGNAL

A value that rises and falls: gauge.

> Quayside's team writes: 99.95% of fare lookups must complete in under 200 milliseconds, measured over 28 days.
>
> It is an internal goal. No customer contract refers to it.

What is this statement?

A An SLO

B An SLI

C An SLA

D An error budget

◆ PAUSE AND DECIDE

What is this statement?

A An SLO

A target for a measurement, used internally.

B An SLI

The measurement itself, without a target.

C An SLA

A contract with consequences for customers.

D An error budget

What the target leaves over: here 0.05%.

THE SIGNAL

Measurement plus target, internal: SLO. Add a contract: SLA.

> The booking API's error rate is already stored in Prometheus.
>
> The team now wants a dashboard that shows it on a wall screen in the operations room.

Which tool draws that dashboard?

A Alertmanager

B The Pushgateway

C Fluent Bit

D Grafana

◆ PAUSE AND DECIDE

Which tool draws that dashboard?

A Alertmanager

Routes alerts; draws no dashboards.

B The Pushgateway

Holds pushed metrics for short jobs.

C Fluent Bit

Ships logs from nodes.

D Grafana

Queries Prometheus and draws dashboards.

THE SIGNAL

Stored in Prometheus, shown on a dashboard: Grafana.

> A CNCF project has production adopters and maintainers from several companies.
>
> It has not yet completed a third-party security audit.

At which maturity level is it most likely?

A Sandbox

B Incubating

C Graduated

D Archived

◆ PAUSE AND DECIDE

At which maturity level is it most likely?

A Sandbox

It has already outgrown the early, experimental stage.

B Incubating

Growing production use, not yet graduated.

C Graduated

Graduation requires the security audit.

D Archived

It is active and growing.

THE SIGNAL

Production users, no audit yet: incubating, not graduated.

Module 30 in five lines

- **Wrong answers are neighbours.** Learn the tiebreaker for each pair.
- **Find the planted detail first:** 'must not restart', 'every node', 'a few users'.
- **Delivery:** blue-green vs canary, prune vs selfHeal, liveness vs readiness.
- **Architecture:** counter vs gauge, SLI vs SLO vs SLA, HPA vs VPA vs node autoscaler.
- **Replay this module** the day before the exam.

Next: how KCNA questions are written, and how to take the exam.

admin@sthithapragna.com